



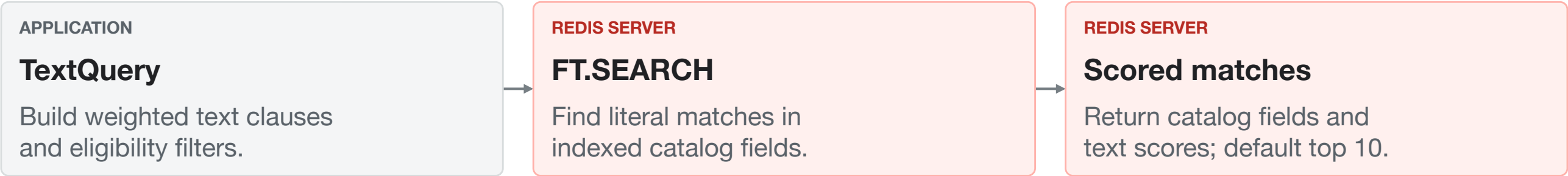
Text, hybrid, and reranked search

Redis retrieves candidates. A local model can change their order without changing the index.

Redis server Local model / RedisVL Application logic Left to right: call order and purpose

01
Text search

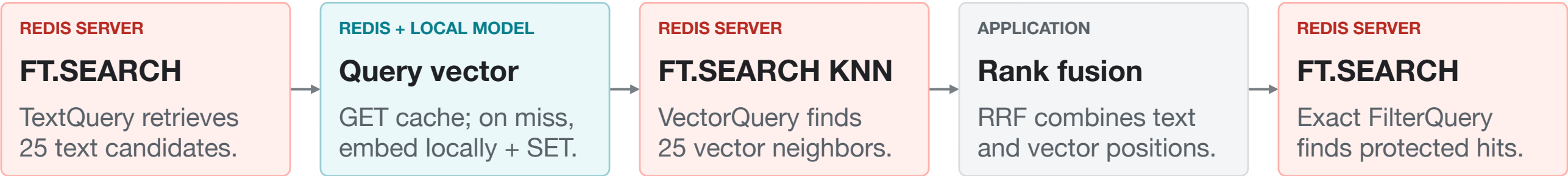
mode: baseline



Why: establish a literal-match baseline. No catalog embedding or cross-encoder is needed.

02
Hybrid search

mode: hybrid



Why: add meaning-based recall, then protect exact matches in the app. RRF is not a Redis command.

03
Reranked search

mode: reranked

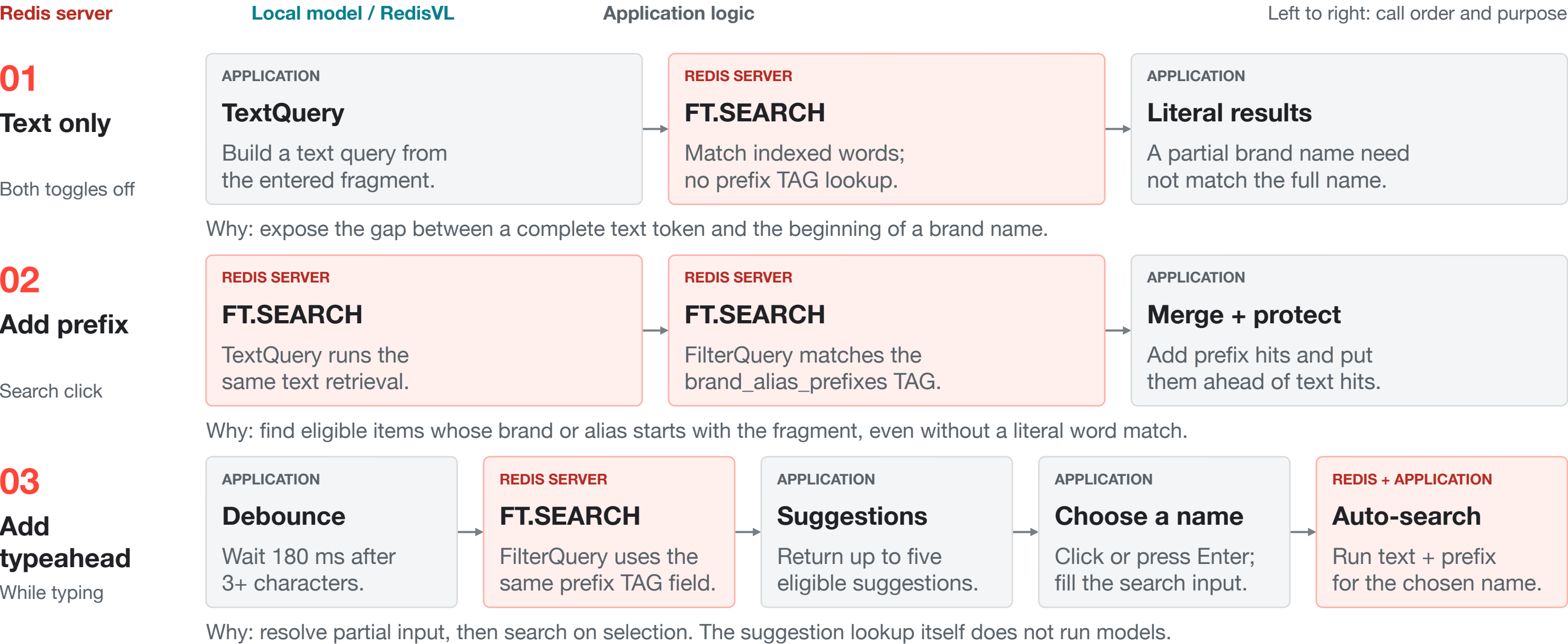


Why: score full query-to-item relevance. Switching rerankers needs no Redis reindexing.



Prefix search and typeahead

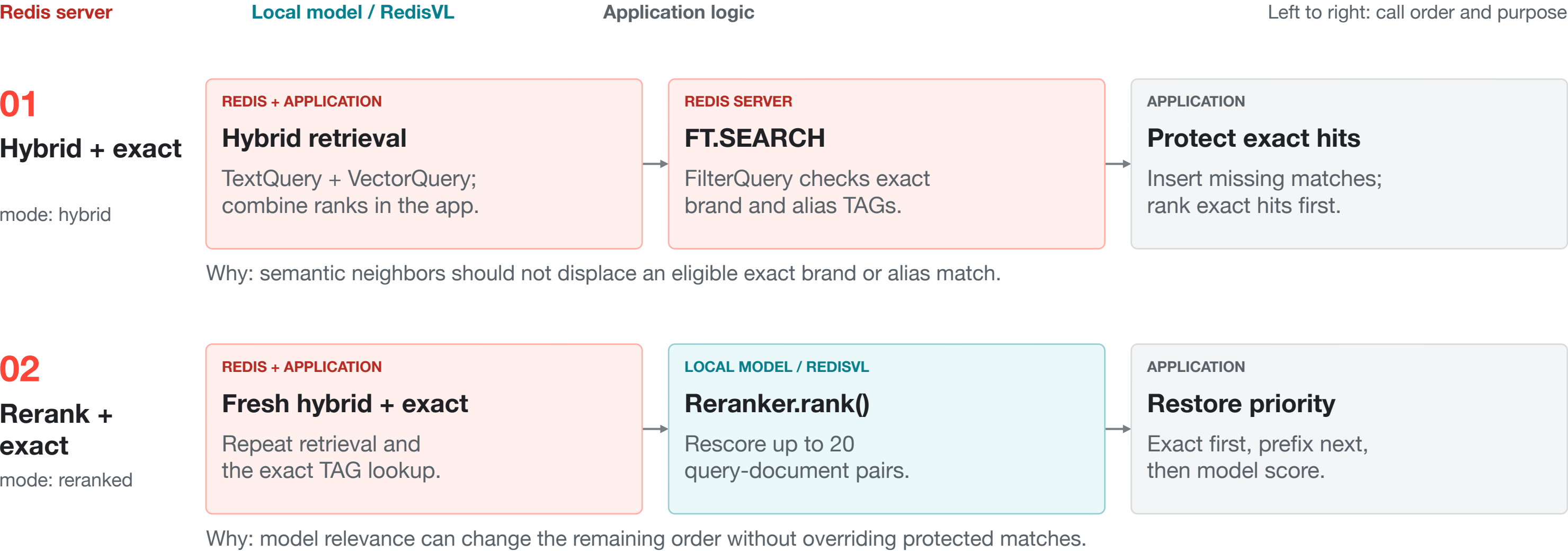
One indexed prefix field supports both result expansion and suggestions while typing.





Exact-match protection

Redis finds exact brand and alias matches. The app preserves their rank priority.

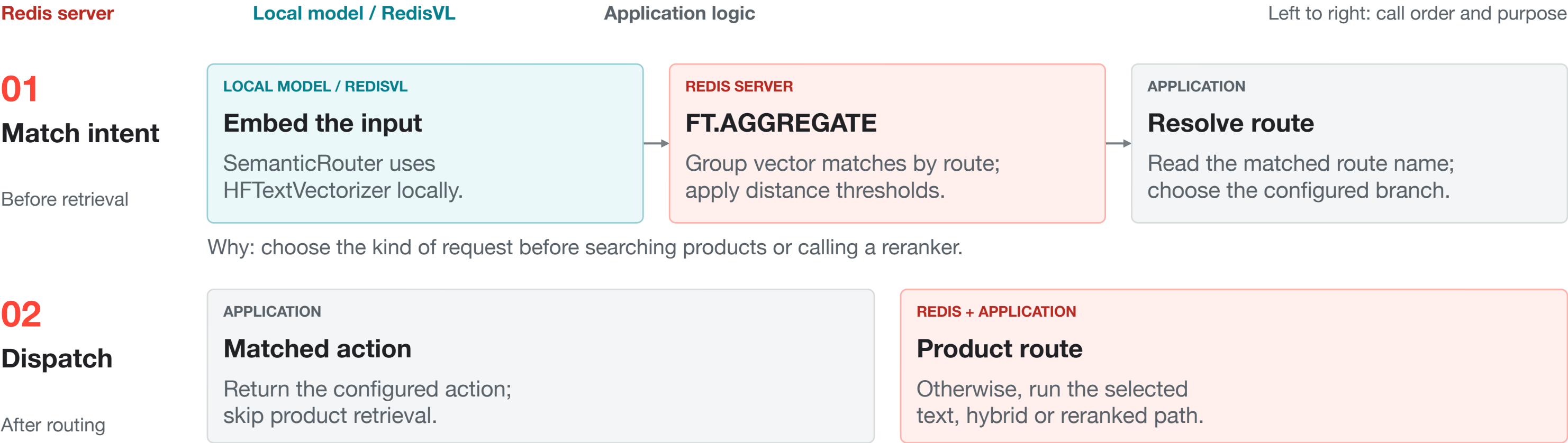


Every catalog lookup still applies tenant and active-item filters. Exact matching does not bypass eligibility.



Semantic routing

Redis stores route reference vectors separately from the product catalog.



Why: choose the kind of request before searching products or calling a reranker.

These are alternative outcomes, not sequential calls. An action does not run the catalog reranker.

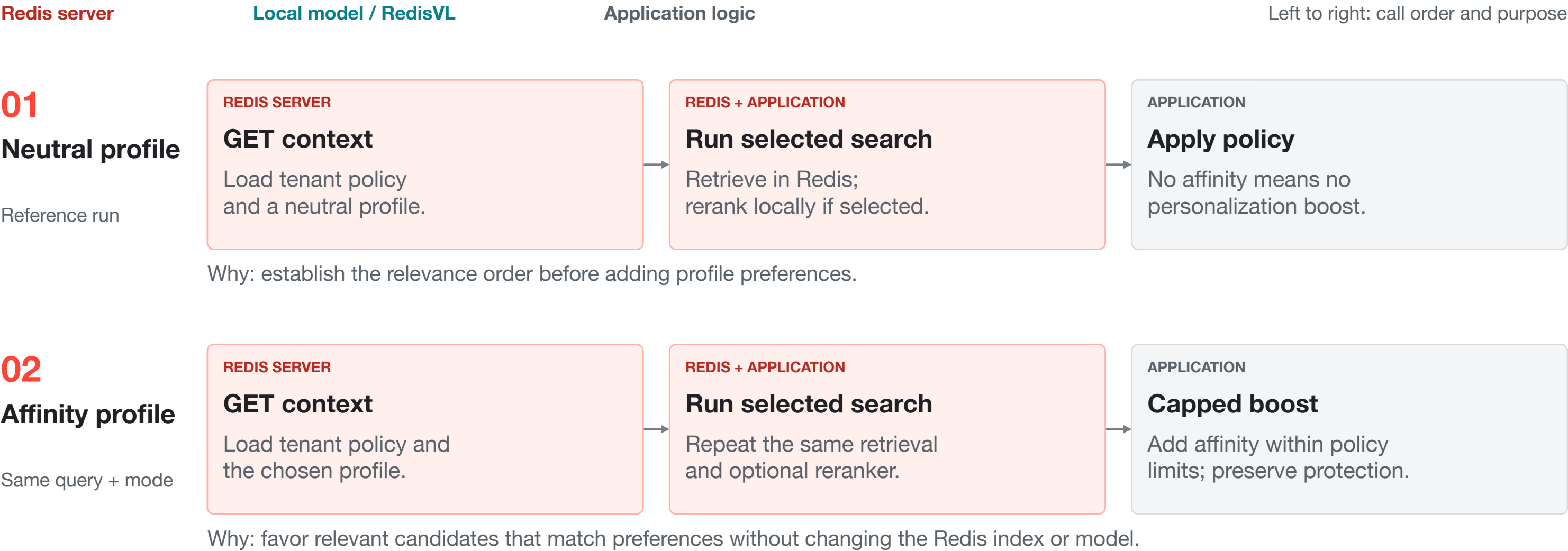
LOW CONFIDENCE OR ROUTER ERROR

Try a configured exact-phrase rule; otherwise continue with product search.



Bounded personalization

Redis supplies profile and policy data. The app applies capped boosts after relevance scoring.



The app can reorder returned candidates, not retrieve new items through a profile boost. Results may stay the same.